Predicting And Preventing 30-Day Hospital Readmissions

4495 Midcourse Progress Report

Le Quang Thien Nguyen – Solo

Student Number: 300354247

CSIS 4495 – 002

Demo Website: https://f26-4495-s2-thien-n.vercel.app/

Midcourse Video Demonstration:
https://drive.google.com/file/d/1OpZZ8iag4pu7Gx0UQpTXcqXPoOGV1NZ-/view?usp=sharing

GitHub repository: https://github.com/ThienNguyen009/F26_4495_S2_ThienN

Submission Date: 26 Oct 2026

Checklist (CHECK EACH ENTRY TO VERIFY THAT IT HAS BEEN DONE)

☒	**GitHub Repository Code Check-In:** Completed GitHub repository check in for all work completed thus far
☒	**Video Demo of the Product:** at least 6 minutes video (per team) showing a demo of all the work completed thus far. The video needs to show a demo of the features, each and every team member must present and demo the work they have each completed. There must be one joint demo of the features (no separate video for each team member). The video must be uploaded to a publicly viewable link (any place of your choice), and the link must be provided in the title page of the MidCourse Progress Report. If the implemented application does not work or your group do not have a video of the demo or the demo does not satisfy the target deliverables and milestones, you may get ZERO mark on your MidCourse Progress evaluation
☒	**Video Link:** Clickable link of the video (link must be clickable and viewable publicly) in the **title page of the MidCourse Progress report**
☒	**Installation Instructions:** Updates to the ReadMe File stating clear project/product description, and a section indication installation instructions. How can I use the repository files, to see a demo of your product? Installation instructions can either be in the README file initially created or as a separate file in the GitHub repository

☒	**MidCourse Progress Report Blackboard Submission:** Completed Report submitted on Blackboard by the Team Lead
☒	**GitHub Repository MidCourse Progress Report Check-in:** Completed Report checked into the GitHub Repository

1.Introduction:

1.1 Background:

A hospital readmission within 30 days of discharge is one of the most watched numbers in healthcare. In the United States, the Medicare Hospital Readmissions Reduction Program (HRRP) reduces payments to hospitals whose 30-day readmission rates are higher than expected. In Canada, the Canadian Institute for Health Information (CIHI) reports the same kind of indicator for every province, including British Columbia.

A readmission costs a lot. HCUP reports that the average readmission in the United States cost about $16,300 in 2020. It also usually means that something in the first discharge did not work, for example the medication list, the follow-up plan or the handover to home care.

Hospitals already collect data that could help: diagnoses, number of conditions, length of stay, previous admissions and where the patient goes after discharge. What most hospitals do not have is a simple, systematic way to use that data. Deciding who needs extra follow-up still depends on a checklist or on a clinician's judgment under time pressure. In this project I treat that gap as an analytics problem. The goal is to turn the discharge record into an automatic risk screen that runs for every patient and is shown in a web application that hospital staff can log in to and use.

1.2 Problem and research questions:

The main question of my project is: **which patient, clinical and administrative factors are most strongly associated with 30-day readmission, and can a machine-learning model flag high-risk patients early enough, and clearly enough, for a discharge team to act before the patient leaves?**

And I created 3 more reasons matter for this reason:

1. **Many readmissions can be prevented.** A follow-up call, a home-care referral or a medication review works when it reaches the right patient. The hard part is knowing who that patient is while they are still in the hospital.
2. **Follow-up capacity is limited.** No hospital can call every patient, so a useful system has to rank patients by risk.
3. **A number alone is not enough.** Staff need to know why a patient was flagged and what to do about it.

From the main question I made four smaller questions:

- Which factors show the strongest association with 30-day readmission?
- Do random forest and XGBoost do better than a logistic regression baseline once class imbalance is handled?

- Does the model work equally well, and give realistic probabilities, for different age, race, sex and diagnosis groups?
- If the model's risk tiers are linked to actions, what is the estimated cost saved compared with the cost of those actions?

1.3 Related work:

Dataset. Strack et al. (2014) introduced the UCI "Diabetes 130-US Hospitals" dataset, which I use in this project. They showed that whether HbA1c was measured during the stay was linked to readmission rates.

Model choice. Studies on this dataset usually find that tree ensembles such as random forest and gradient boosting score higher than logistic regression. However, logistic regression is still useful as an easy-to-read baseline to compare against. Prior inpatient admissions usually predict readmission better than individual diagnosis codes.

Class imbalance. Readmitted patients are a small part of all discharges (about 11% in my cleaned data). A model trained without any correction learns to predict "not readmitted" for everyone. The two common fixes are class weighting and SMOTE, the synthetic oversampling method from Chawla et al. (2002).

Explainability. Clinicians say a black-box score is hard to act on. SHAP values (Lundberg & Lee, 2017) are the standard way to show which inputs pushed a prediction up or down, for all patients and for one patient.

Interventions. A meta-analysis of randomized trials by Leppin et al. (2014) found that discharge interventions reduced 30-day readmission with a pooled relative risk of 0.82, and that more complete interventions worked better.

The gap. Most student and course projects stop at the risk score. They rarely connect the score to a specific action, estimate what that action saves, check performance across patient groups, or put the result in a secure tool that staff could use. My project tries to cover all of these.

1.4 Hypothesis, assumptions and benefits:

Hypotheses:

- H1: Prior inpatient admissions will be one of the strongest single predictors. My exploratory analysis already supports this (univariate AUC 0.613).
- H2: The ensemble models will beat logistic regression, but only by a small amount.
- H3: Class correction will push predicted probabilities too high, so calibration will be needed before the cost model uses them.

Assumptions and limitations: the data covers diabetic inpatients in US hospitals from 1999 to 2008, so my results show a method rather than numbers that transfer directly to BC today. The data is public and de-identified, so no ethics review is needed. The rule-based actions for each risk tier stand in for a real clinical protocol.

Potential benefit: the finished project shows how a hospital could use data it already has to build a low-cost, explainable screening step, and what that step might be worth compared with the cost of preventable readmissions.

2. Summary of the Initially Proposed Research Project:

In my proposal I planned a quantitative, predictive-modelling study with three layers:

- **Exploratory layer:** measure how each factor relates to readmission.
- **Modelling layer:** build a logistic regression baseline, then compare random forest and XGBoost against it.
- **Application layer:** turn the model output into an explainable decision for each patient, shown in a web app.

Objectives:

1. Quantify the association between each candidate predictor and 30-day readmission.
2. Build a logistic regression baseline and compare it with random forest and XGBoost, correcting for class imbalance.
3. Use SHAP for global feature importance and per-patient explanations.
4. Test whether performance and calibration hold across demographic and diagnosis groups.
5. Turn the output into three risk tiers, each linked to an action and an estimated cost avoided.
6. Deliver everything as a web application: a React front end over a FastAPI machine-learning service.

Data: the UCI "Diabetes 130-US Hospitals for Years 1999–2008" dataset, with 101,766 encounters from 130 hospitals and about 50 features each. Because many patients have more than one encounter, the data is split by patient so that no patient appears in more than one of the train, validation and test sets.

Prediction timing: discharge disposition and discharge medications are only known when the discharge order is written, so the tool predicts at that moment, not at admission.

Proposed technology:

Layer	Technology
Languages	Python 3.11, TypeScript, SQL
Data	PostgreSQL with an encounters fact table, dimension tables and SQL views
Modelling	pandas, scikit-learn, imbalanced-learn, XGBoost, SHAP
Back end	FastAPI with SQLAlchemy and Pydantic
Front end	React with TypeScript (Vite) and Recharts, five screens
Tooling	Git and GitHub, GitHub Projects Kanban, pytest and Vitest, Docker Compose

Original timeline: six phases: setup (weeks 1–2), data cleaning and EDA (3–4), baseline modelling (5–6), ensemble modelling and tuning (7–9), explainability and API (10–11), and web application and write-up (12–13).

3. Changes to the Proposal:

3.1 New Features: user login with password protection:

Change: I added user accounts to the web application. Users must log in with a username and password before they can see any patient risk scores, plus I also added user input so I can collect data from user then add it into my dataset too. This was not in my original proposal.

Justification: the app shows a risk score, the reasons behind it and a recommended action for individual patients. My dataset is de-identified, but in a real hospital this screen would show personal health information, and anyone handling that kind of data has to control who can see it. Without a login, anyone with the link could open the patient list. I also wanted the project to look like a tool a hospital could deploy, not only a demo.

Technology added: JavaScript, Python, FastAPI's security tools, PostgreSQL

Impact on the timeline: this took about 10-14 hours last week around 21st to now 26 to do it, I finished it without moving any modelling milestone. I saved it as milestone 2.

3.2 Data cleaning: removing patients who died or went to hospice:

Change: my proposal only said I would drop encounters with no recorded discharge disposition. I also removed encounters where the patient died or was discharged to hospice (disposition codes 11, 13, 14, 19, 20 and 21), and the few rows with an invalid gender value.

Justification: a patient who died or went to hospice cannot be readmitted, so keeping them would make the model's job look easier than it really is and inflate the results. Removing them is the standard approach for this dataset. After all cleaning steps, the dataset went from 101,766 to 94,660 encounters, and the 30-day readmission rate went from 11.2% to 11.4%.

3.3 One shared cleaning module for training and the api:

Change: instead of cleaning the data inside notebooks, I put all cleaning and feature-engineering steps into one Python module that both the training code and the FastAPI service use.

Justification: if the API cleaned data with copied code, the two versions could slowly drift apart and the app would give wrong predictions without any error message. Using one module means the app always prepares data exactly the same way as during training.

4. Project Planning and Timeline:

I am working alone, so I am responsible for every part of the project: data engineering, modelling, back end, front end and reporting. The table below shows the plan from now until the end of the term.

Completed so far (weeks 1–6):

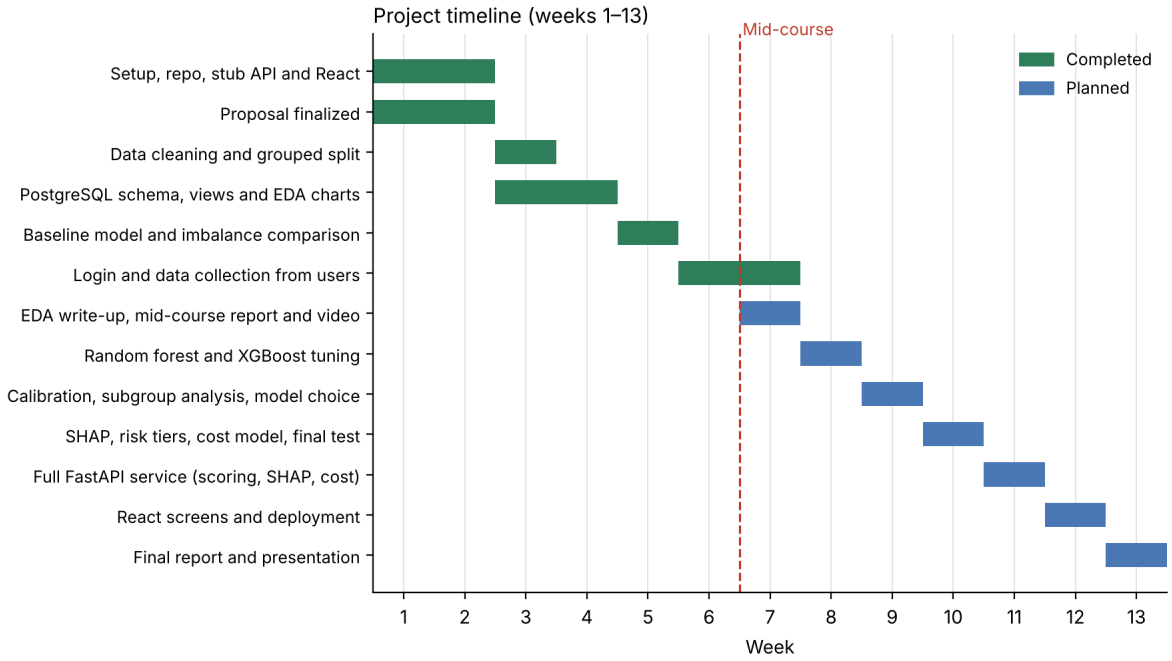
Weeks	Work completed	Status
1–2	Proposal; GitHub repository; Python environment; PostgreSQL in Docker; starter FastAPI and React apps	Done
3	Data profiling; cleaning and feature-engineering module; patient-grouped 60/20/20 split; unit tests	Done
3–4	Star-schema database; SQL views; 7 EDA charts; 27 features ranked by univariate AUC	Done
4-5	Logistic regression baseline: no correction, class weighting and SMOTE compared, Baseline write-up finish the written EDA findings; read the logistic regression coefficients; record the mid-course video; in-class check-in	Done
6	User login with password (FastAPI + JWT); logged-in users can submit patient records, which are saved to the database	Done

From week 7:

Week	Phase	Tasks	Deliverable
7	Check login, password, fix the frontend		MidCourse report and video
7	Ensemble models	Train random forest and XGBoost; tune them with patient-grouped cross-validation; compare with the baseline	Model comparison table
8	Calibration	Check calibration curves and Brier scores; apply Platt scaling or isotonic regression	Calibrated model
9	Fairness and model choice	Recompute AUC, recall and calibration by age, race, sex and diagnosis group; choose the final model	Subgroup report; final model choice
10	Explainability and tiers	SHAP explanations; set three risk tiers; cost-benefit model with sensitivity analysis; one final test-set evaluation; save the model	Saved model and report file
11	API	Scoring, CSV upload, single-patient prediction with SHAP reasons, metrics and cost endpoints, all behind the login; pytest tests	Working, tested API

Week	Phase	Tasks	Deliverable
12	Web app	Login page plus the five screens; Vitest tests; deploy with Docker to a free host	Live web app
13	Write-up	Final report, presentation and live demo	Final report and presentation

Project management chart. I track tasks on a GitHub Projects Kanban board (Backlog, In Progress, Review, Done). The Gantt chart below shows the same plan by week.



5. Implemented Features:

This is a solo project, so all the features below were designed and built by me.

5.1 Data cleaning and feature engineering pipeline:

I started by profiling the raw data: 101,766 encounters, about 50 columns, and many patients with more than one encounter. I found three main problems: the target is imbalanced (11.2% readmitted within 30 days), some columns are mostly missing (for example weight), and repeat patients could leak information between the training and test sets.

The cleaning module does the following:

- Defines the target: readmitted within 30 days = 1, everything else = 0.
- Removes encounters that cannot be readmitted (death or hospice), encounters with no recorded disposition, and invalid gender values.

- Treats "?" as missing, and keeps "None" in the HbA1c and glucose test columns as a real value meaning "test not done".
- Groups more than 700 ICD-9 diagnosis codes into 9 clinical groups, and groups the discharge, admission and specialty codes into a few categories.
- Summarizes the 23 diabetes medication columns into two numbers: how many diabetes drugs the patient got, and how many had a dose change.
- Adds features such as total prior visits and a numeric age.

The result is a clean dataset of 94,660 encounters with an 11.4% readmission rate.

```
 97
 98   def add_target(df: pd.DataFrame) -> pd.DataFrame:
 99       df = df.copy()
100       df["target"] = (df["readmitted"] == "<30").astype(int)
101       return df
102
```

PROBLEMS JUPYTER TERMINAL OUTPUT DEBUG CONSOLE

TERMINAL

```
(.venv) thiennguyen@Thiens-MacBook-Air ml % python3 features.py
  return pd.read_csv(path, na_values=["?", ""], keep_default_na=False)
Raw rows:      101,766
Removed:       7,106
Clean rows:    94,660
Columns:       62
Readmit rate:  11.4%
(.venv) thiennguyen@Thiens-MacBook-Air ml %
```

This is the build_features function :

```
def build_features(df: pd.DataFrame) -> pd.DataFrame:
    """Turn raw UCI-format rows into model features. Never drops rows."""
    df = df.copy()
```

This is for groupings, to make sure its easier to classify based on groups:

```
### this code groupings:

    df["disposition_group"] =
df["discharge_disposition_id"].map(DISPOSITION_GROUPS).fillna("Other")
    df["admission_type"] =
df["admission_type_id"].map(ADMISSION_TYPE_GROUPS).fillna("Other")
    df["admission_source"] =
df["admission_source_id"].map(ADMISSION_SOURCE_GROUPS).fillna("Other")
    for d in ["diag_1", "diag_2", "diag_3"]:
        df[f"{d}_group"] = df[d].apply(icd9_group)
```

5.2 Patient-grouped trained, validation and test split:

I split the clean data into 60% training, 20% validation and 20% test. The split is grouped by patient ID and stratified on the target, so each patient appears in only one set and each set has almost the same

readmission rate. This matters because a normal random split would put the same patient in both training and test data, which would make the model look better than it really is.

I wrote unit tests with pytest that check that no patient appears in two sets and that no deceased or hospice patients remain. All tests pass.

This is for the split code and the pytest output showing all tests passing.

```
import pandas as pd
from sklearn.model_selection import StratifiedGroupKFold

df = pd.read_parquet("data/clean.parquet")

####  1. Hold out 20% as  the test
outer = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=42)
trainval_pos, test_pos = next(outer.split(df, df["target"], groups=df["patient_nbr"]))

df["split"] = "train"
df.iloc[test_pos, df.columns.get_loc("split")] = "test"

### 2. From the remaining 80%, just for hold out a quarter (like = 20% of the total)
as validation
trainval = df.iloc[trainval_pos]
inner = StratifiedGroupKFold(n_splits=4, shuffle=True, random_state=42)
_, val_pos = next(inner.split(trainval, trainval["target"],
groups=trainval["patient_nbr"]))
df.loc[trainval.index[val_pos], "split"] = "val"

print(df.groupby("split")["target"].agg(["size", "mean"]))
df.to_parquet("data/clean_split.parquet", index=False)
```

5.3 PostgreSQL Database:

I set up PostgreSQL in Docker with Docker Compose and designed a star schema: one encounters fact table plus dimension tables for patients, discharge disposition, admission type and admission source. I loaded the cleaned data into it and wrote SQL views that calculate the readmission rate by prior inpatient stays, discharge disposition, age band, length of stay, medication count, diagnosis group, admission type and race. These views use the training data only, so the test set does not affect my analysis. The same views could also be connected to Power BI or Tableau later.

```
thiennguyen@Thiens-MacBook-Air ml %  source /Users/thiennguyen/Desktop/F26_4495_S2_ThienN/.venv/bin/activate
(.venv) thiennguyen@Thiens-MacBook-Air ml % docker compose up -d
docker compose exec db psql -U readmit -d readmit
[+] up 1/1
 ✔ Container readmission-db-1 Running                                                                    0
psql (16.15 (Debian 16.15-1.pgdg13+2))
Type "help" for help.

readmit=# \dt
              List of relations
 Schema |         Name          | Type  |  Owner
--------+-----------------------+-------+---------
 public | dim_admission_source  | table | readmit
 public | dim_admission_type    | table | readmit
 public | dim_disposition       | table | readmit
 public | dim_patient           | table | readmit
 public | encounters            | table | readmit
 public | sessions              | table | readmit
 public | users                 | table | readmit
(7 rows)

readmit=#
```

PROBLEMS JUPYTER TERMINAL OUTPUT DEBUG CONSOLE

TERMINAL

```
                    Table "public.encounters"
          Column           |  Type   | Collation | Nullable | Default
---------------------------+---------+-----------+----------+---------
 encounter_id              | bigint  |           | not null |
 patient_nbr               | bigint  |           |          |
 split                     | text    |           | not null |
 age_band                  | text    |           |          |
 age_num                   | integer |           |          |
 admission_type_id         | integer |           |          |
 admission_source_id       | integer |           |          |
 discharge_disposition_id  | integer |           |          |
 time_in_hospital          | integer |           |          |
 num_lab_procedures        | integer |           |          |
 num_procedures            | integer |           |          |
 num_medications           | integer |           |          |
 number_outpatient         | integer |           |          |
 number_emergency          | integer |           |          |
 number_inpatient          | integer |           |          |
 number_diagnoses          | integer |           |          |
 diag_1_group              | text    |           |          |
 diag_2_group              | text    |           |          |
 diag_3_group              | text    |           |          |
 a1c_result                | text    |           |          |
 max_glu_serum             | text    |           |          |
 insulin                   | text    |           |          |
 n_diabetes_meds           | integer |           |          |
 n_med_changes             | integer |           |          |
 total_prior_visits        | integer |           |          |
 target                    | integer |           | not null |
Indexes:
    "encounters_pkey" PRIMARY KEY, btree (encounter_id)
Foreign-key constraints:
    "encounters_discharge_disposition_id_fkey" FOREIGN KEY (discharge_disposition_id) REFERENCES dim_disposition(d
isposition_id)
    "encounters_patient_nbr_fkey" FOREIGN KEY (patient_nbr) REFERENCES dim_patient(patient_nbr)

(END)
```

```
               List of relations
 Schema |         Name          | Type  |  Owner
--------+-----------------------+-------+---------
 public | dim_admission_source  | table | readmit
 public | dim_admission_type    | table | readmit
 public | dim_disposition       | table | readmit
 public | dim_patient           | table | readmit
 public | encounters            | table | readmit
 public | sessions              | table | readmit
 public | users                 | table | readmit
(7 rows)

readmit=# \dv
                 List of relations
 Schema |           Name            | Type |  Owner
--------+---------------------------+------+---------
 public | v_base_rate               | view | readmit
 public | v_rate_by_admission_type  | view | readmit
 public | v_rate_by_age             | view | readmit
 public | v_rate_by_diag            | view | readmit
 public | v_rate_by_disposition     | view | readmit
 public | v_rate_by_los             | view | readmit
 public | v_rate_by_meds            | view | readmit
 public | v_rate_by_prior_inpatient | view | readmit
 public | v_rate_by_race            | view | readmit
 public | v_train                   | view | readmit
(10 rows)

readmit=#
```

```
readmit=# SELECT * FROM v_rate_by_prior_inpatient;
 prior_inpatient |   n   | readmit_rate
-----------------+-------+--------------
 0               | 37826 |       0.0838
 1               | 10884 |       0.1310
 2               |  4145 |       0.1812
 3               |  1839 |       0.2077
 4+              |  2158 |       0.3197
(5 rows)

readmit=#
```

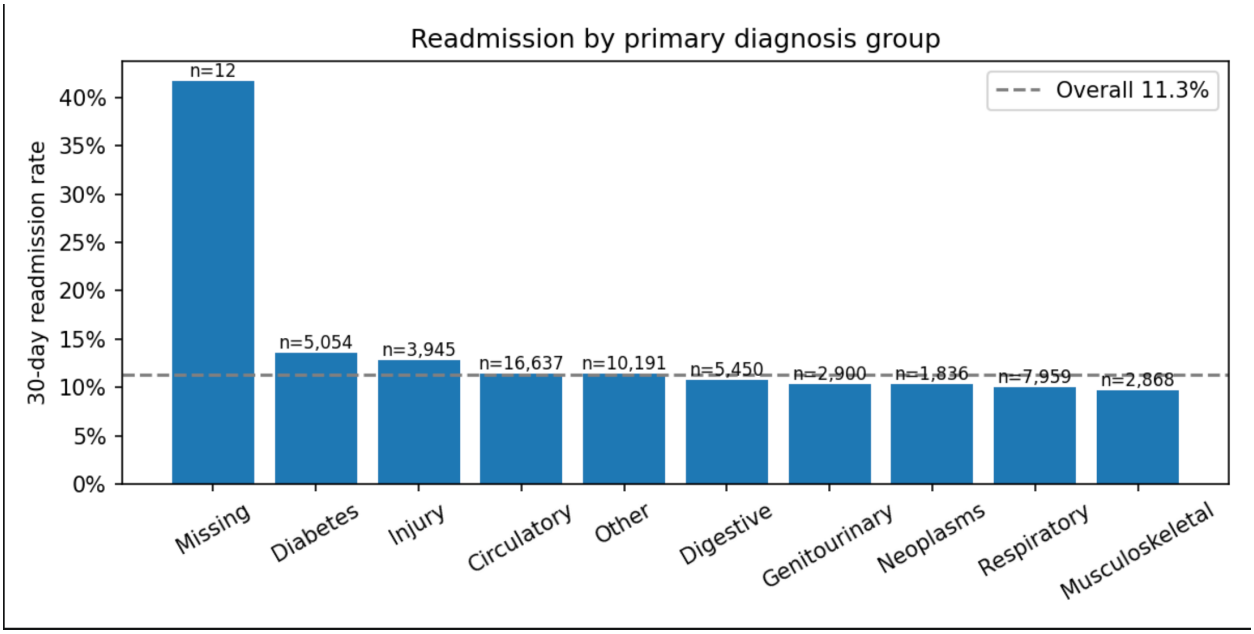
5.4 Exploratory Data Analysis:

Using the SQL views, I made 7 charts of readmission rates by key factors and ranked 27 features by univariate AUC (how well each feature on its own separates readmitted from non-readmitted patients; 0.5 means no signal).

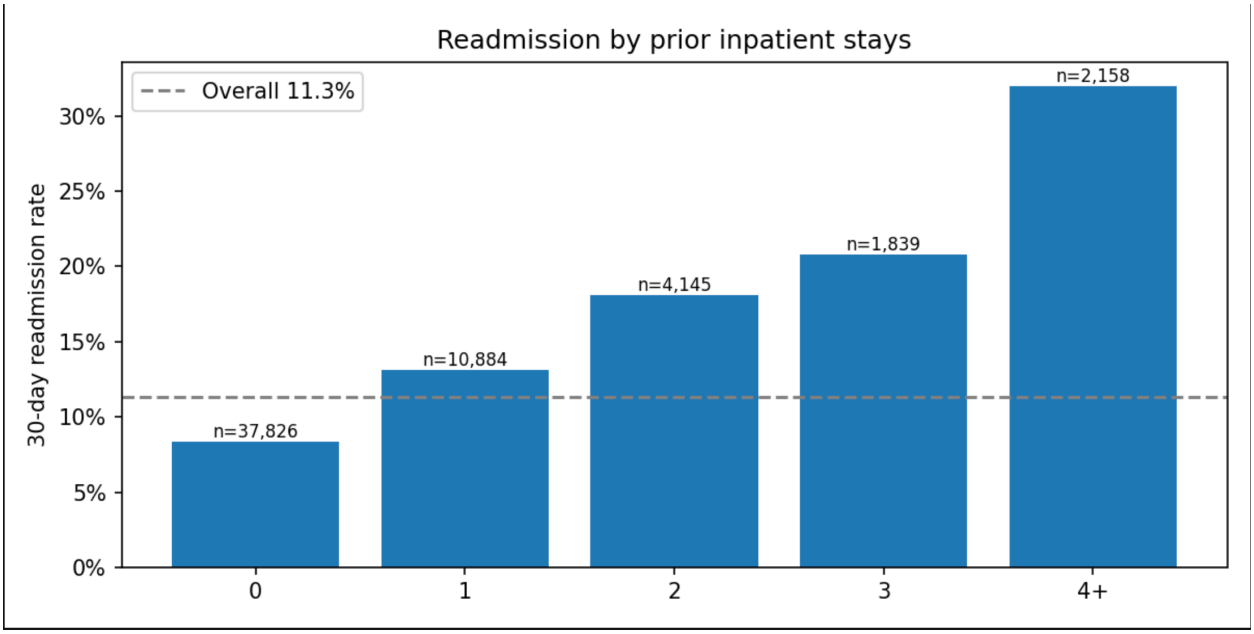
- **Prior inpatient stays** is the strongest predictor (AUC 0.613). The readmission rate rises from 8.4% for patients with no prior stays to 32.0% for patients with four or more.
- **Total prior visits** and **discharge disposition** come next.
- **Race and gender** show almost no predictive signal on their own. I still keep them for the fairness check in week 9.

No single factor separates the two groups clearly, so the next step is to combine them in a model.

Here this is a chart for comparing of readmission based on primary diagnosis group:



And this is for comparing by prior inpatient stays:



5.5 Baseline model and class imbalance comparison:

I trained logistic regression three ways, using a scikit-learn pipeline that scales the numeric columns and one-hot encodes the categories:

- **No correction**, as a reference.
- **Class weighting**, my main method from the proposal: mistakes on readmitted patients count more.
- **SMOTE**, the alternative: creates synthetic readmitted patients, only inside the training data.

```
models = {
    # this "No correction:" shows the "predict no for everyone" trap
    "LR (no correction)": Pipeline([("prep", make_preprocessor()),
                                    ("model", LogisticRegression(max_iter=2000))]),
    # and Primary method from the proposal
    "LR (class weights)": Pipeline([("prep", make_preprocessor()),
                                    ("model", LogisticRegression(class_weight="balanced", max_iter=2000))]),
    # Alternative: SMOTE, applied inside the pipeline so it only touches training folds
    "LR (SMOTE)": ImbPipeline([("prep", make_preprocessor()),
                               ("smote", SMOTE(random_state=42)),
                               ("model", LogisticRegression(max_iter=2000))]),
}
```

All three were compared on the validation set:

	model	roc_auc	pr_auc	precision_at_top20	recall_at_0.5	precision_at_0.5	flagged_share_at_0.5	brier	mean_predicted	base_rate
0	LR (no correction)	0.6448	0.1953	0.1936	0.0117	0.5000	0.0027	0.0981	0.1101	0.114
1	LR (class weights)	0.6458	0.1944	0.1933	0.5192	0.1730	0.3421	0.2249	0.4628	0.114
2	LR (SMOTE)	0.6316	0.1902	0.1936	0.5136	0.1676	0.3494	0.2243	0.4551	0.114

The real readmission rate in the validation set is 11.4%, so a PR-AUC around 0.19 is well above what random guessing would give (0.114).

All three models rank patients about equally well (ROC-AUC 0.63 to 0.65). But the model with no correction is almost useless in practice: at a 0.5 cut-off it flags only 0.3% of patients and catches just 1.2% of the patients who were readmitted. Class weighting and SMOTE both raise recall to about 52%. I chose **class weighting** because it scores as well as or slightly better than SMOTE (ROC-AUC 0.646 vs 0.632, PR-AUC 0.194 vs 0.190), it is simpler, and it trains only on real patients instead of synthetic ones.

One problem shows up in the last column. Both corrected models predict an average risk of about 46%, while the real rate is 11.4%, so their probabilities are much too high. This is expected with class correction, and it matters because the cost model will multiply these probabilities by dollar amounts. I will fix it with calibration in week 9.

5.6 React front end and login page :

The front end is a React and TypeScript app created with Vite. It now has a login page where the user enters a username and password.

Once logged in, the user can fill in a form to submit a patient record, which is saved to the database, and can request a prediction from the API and see the result on the page.

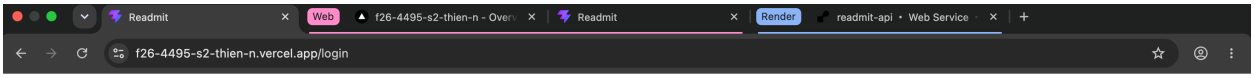
This is the interface where you can create an account by clicking the create one, or if u have an account, you just type it in here and get into the page

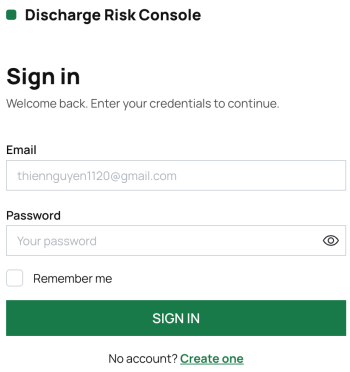
I created one for you to test:

Email: quangthiennguyen1120@gmail.com

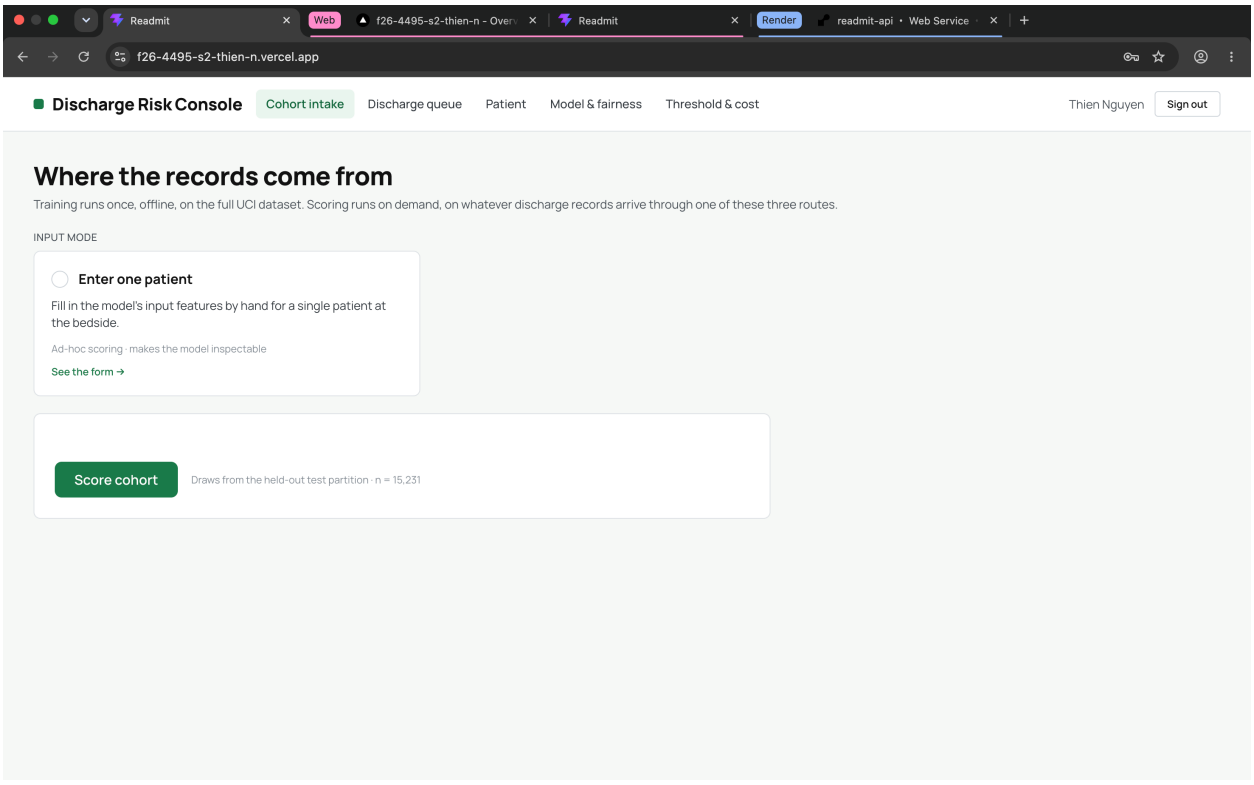
Password: Thien@1234567

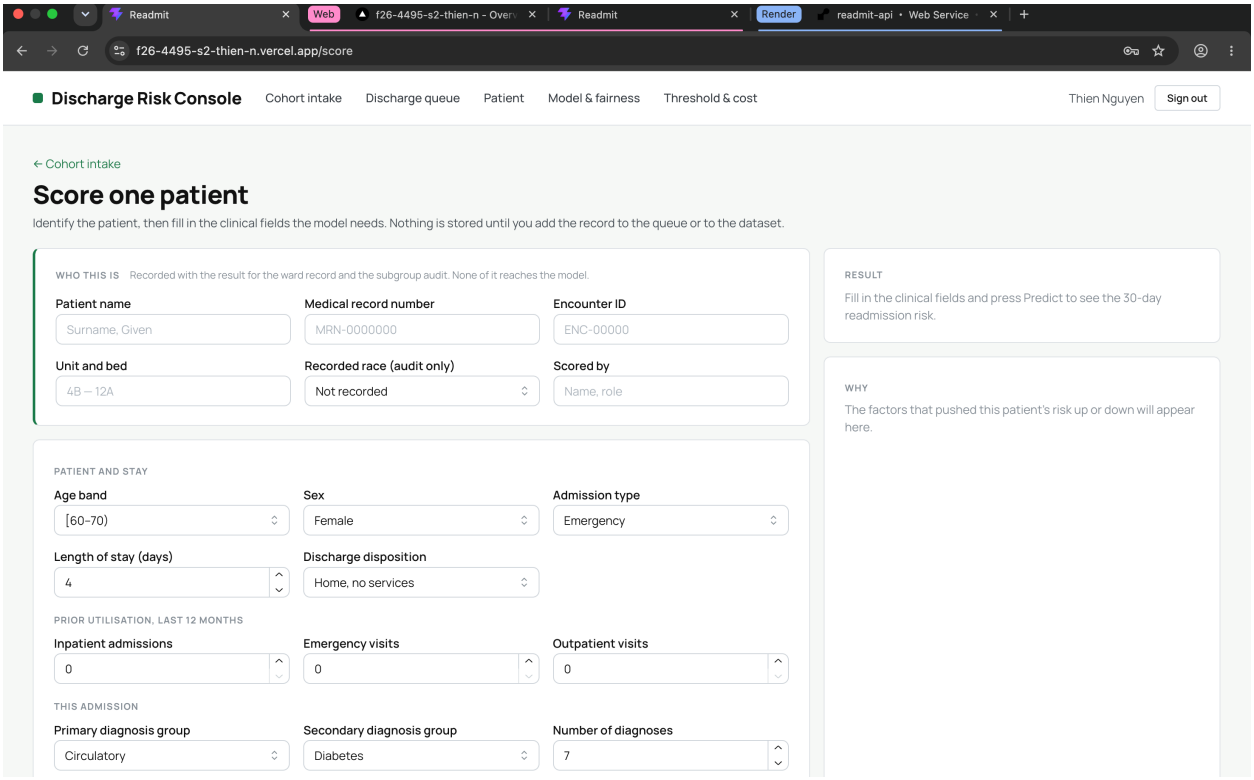
And also after filling out the form, u can save it directly into the dataset too.

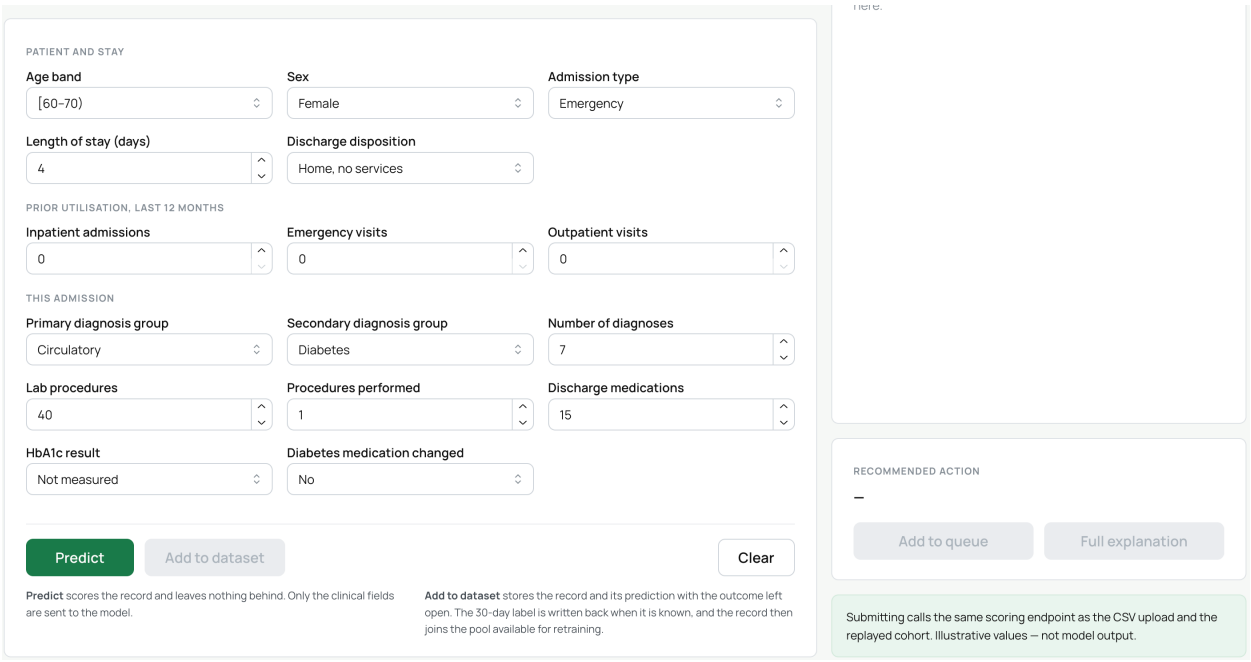




This is the page:


Readmit
Web
f26-4495-s2-thien-n - Overv
Readmit
Render
readmit-api • Web Service
f26-4495-s2-thien-n.vercel.app
Discharge Risk Console
Cohort intake
Discharge queue
Patient
Model & fairness
Threshold & cost
Thien Nguyen
Sign out
Where the records come from
Training runs once, offline, on the full UCI dataset. Scoring runs on demand, on whatever discharge records arrive through one of these three routes.
INPUT MODE
Enter one patient
Fill in the model's input features by hand for a single patient at the bedside.
Ad-hoc scoring · makes the model inspectable
See the form →
Score cohort
Draws from the held-out test partition · n = 15,231


Readmit
Web
f26-4495-s2-thien-n - Overv
Readmit
Render
readmit-api • Web Service
f26-4495-s2-thien-n.vercel.app/score
Discharge Risk Console
Cohort intake
Discharge queue
Patient
Model & fairness
Threshold & cost
Thien Nguyen
Sign out
← Cohort intake
Score one patient
Identify the patient, then fill in the clinical fields the model needs. Nothing is stored until you add the record to the queue or to the dataset.
WHO THIS IS
Recorded with the result for the ward record and the subgroup audit. None of it reaches the model.
Patient name
Surname, Given
Medical record number
MRN-0000000
Encounter ID
ENC-00000
Unit and bed
4B — 12A
Recorded race (audit only)
Not recorded
Scored by
Name, role
PATIENT AND STAY
Age band
[60-70)
Sex
Female
Admission type
Emergency
Length of stay (days)
4
Discharge disposition
Home, no services
PRIOR UTILISATION, LAST 12 MONTHS
Inpatient admissions
0
Emergency visits
0
Outpatient visits
0
THIS ADMISSION
Primary diagnosis group
Circulatory
Secondary diagnosis group
Diabetes
Number of diagnoses
7
RESULT
Fill in the clinical fields and press Predict to see the 30-day readmission risk.
WHY
The factors that pushed this patient's risk up or down will appear here.



6. AI tools:

AI Tool Name	Version, Account Type	Specific feature for which the AI tool was used	Value Addition (written by me)
Claude	Plus	Which model is good for my project lie XGBOOST, Linear Regression, and compare these models	I was finding the best model for my project then apply it
Claude	Plus	**Why SMOTE needs ImbPipeline:** a normal scikit-learn Pipeline can't hold a step that adds rows. Imbpipeline can, and it only adds fake patients while **training**, never when scoring validation or test data.	I knew it is for keeping my results honest
Claude	Plus	Explanation of the IDS_mapping.csv codes and which discharge codes to remove	Some key words that I don't understand so I used it to get the explanations for the key words

AI Tool Name	Version, Account Type	Specific feature for which the AI tool was used	Value Addition (written by me)
Claude	Plus	For the baseline, I need to implement logistic regression for 3 models class weight and SMOTE and no correction	I need to understand these 3 models, and get to know more about the main one: class weight and SMOTE as my alternation model
Claude	Plus	How can I add login user and password for the interface	Due to our plan is changed cuz u told me to do user input
Claude	Plus	Interface for the front end, can u pls give me some examples	I got the website call matine.dev with this u can create your website, structure like size, color, style for the frontend
Claude	Plus	Find ways to put the project into any server that I can run it or people who got the link can access it	I got the vercel.com which is to be a server so I can put my front end there for you to see the interface
Claude	Plus	It can't run, can u pls show me is there any way to do the samething for the backend so I can run my project on the website	Dashboard.Render.com is for integrate the backend so people can run and see it
Claude	Plus	Get results from terminal for the sql	So I got the results, then I took a pic of all of it

7. Work date/ hours log:

Student name: Le Quang Thien Nguyen

Here this is the table after the progress report 1:

Date	Hours	Description of work done
14 Oct	3	Compared Streamlit/Power BI/Tableau with a single web app; chose React + FastAPI + PostgreSQL
17 Oct	2	UI mockups of the five application screens

Date	Hours	Description of work done
Oct 19	3	Rewrote the proposal for clarity; updated the timeline for the new stack, because I need to add login, password and collecting user input
Oct 20	4	Methodology review: grouped split, calibration, race in subgroup analysis, caveats, contingency plan
Oct 20	0.45	Set up PostgreSQL in Docker with docker-compose.yml and tested the connection
Oct 21	3	Trained logistic regression with no correction, class weighting and SMOTE; compared them on the validation set; saved model_comparison.csv
Oct 21	2	Plotted ROC and precision-recall curves; chose class weighting
Oct 21	4	Designed the users table and the submitted-records table in PostgreSQL
Oct 22	7	I built the register and login endpoints with password hashing and JWT tokens
Oct 23	~9.5	Protected the prediction and data-submission endpoints with a token check
Oct 24	6	I keep building the interface for only login and password
Oct 25	9	Built the React login page; saved the token and sent it with each request; redirect to login when the token expires
Oct 26	8	Tested the login flow (correct and wrong password, expired token) and data submission
Total		

8. References:

Building the frontend. https://vercel.com

Bulding the backend. https://dashboard.render.com

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. Journal of Artificial Intelligence Research, 16, 321–357.

Jiang, H. J., & Hensche, M. (2023). Characteristics of 30-day all-cause hospital readmissions, 2016–2020 (HCUP Statistical Brief #304). Agency for Healthcare Research and Quality. https://hcup-us.ahrq.gov/reports/statbriefs/sb304-readmissions-2016-2020.jsp

Leppin, A. L., Gionfriddo, M. R., Kessler, M., et al. (2014). Preventing 30-day hospital readmissions: A systematic review and meta-analysis of randomized trials. JAMA Internal Medicine, 174(7), 1095–1107.

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems, 30.

Strack, B., DeShazo, J. P., Gennings, C., Olmo, J. L., Ventura, S., Cios, K. J., & Clore, J. N. (2014). Impact of HbA1c measurement on hospital readmission rates: Analysis of 70,000 clinical database patient records. BioMed Research International, 2014, 781670.

UCI Machine Learning Repository. (2014). Diabetes 130-US hospitals for years 1999–2008. https://archive.ics.uci.edu/dataset/296/diabetes+130-us+hospitals+for+years+1999-2008

NOTE THAT A MIDCOURSE PROGRESS GRADE WILL NOT BE GIVEN WITHOUT the MANDATORY IN-PERSON after submission check in done during CLASS HOURS. All Students must be present for the mandatory check in. The grading rubric for MidCourse Progress Video, Report and Implementation (Repo Check In + In person MidCourse Progress check in)

Criteria	Grading
Report Adherence • Completed CHECKLIST Given in Pg. 1 • The report addresses all the sections mentioned in the template with sufficient clarity and details • Changes to the original proposal is clearly outlined with revised timelines and scope • **EACH STUDENT CONTRIBUTION:** Contribution, in terms of implementation details and work logs are clear for each student • The report is written in good English with appropriate grammatical usages • The report is well rounded in providing information verbally and visually as appropriate	4 points
Video Recording • The video link is provided in the title page of the MidCourse Progress report, is clickable, accurate and viewable without needing any login. The video is a single composite video (only one video per team) of the product features • The video clearly demonstrates features implemented thus far • **EACH STUDENT CONTRIBUTION:** Each Student has contributed towards the team video demonstration • Each student's contribution is clearly demonstrated in the video	4 points
GitHub Repository CODE and installation instructions Check-IN **(evaluated individually for EACH STUDENT)** • Updated the README to include installation instructions (or have installation instructions as a separate file in the repo) • The consistency of code check-in, complexity of code implemented, technical merit of features implemented thus far	7 points